DKD Interview Guide

Simple, natural answers for Tejas

Dimaag Ka Darpan is a privacy-aware journaling backend. This guide is based on the project as it exists now, including dashboard analytics, consent-based AI, weekly reports, Kafka-ready email processing, Redis caching, MongoDB, JWT security, and the API contract used by a separate client.

Your aim Speak honestly and clearly. Do not memorize every word.	**Best answer style** Problem - decision - result - future improvement.

Use this like a conversation guide. Understand the idea first; then explain it in your own words.

1. Your 60-second project introduction

Start with this answer whenever an interviewer asks, “Tell me about your project.”

Give me your project introduction.

DKD, or Dimaag Ka Darpan, is a secure journaling and reflection platform. Users can create private journal entries, choose a sentiment, search their own entries, view dashboard and weekly analytics, and receive a weekly reflection email. I built the backend with Spring Boot, MongoDB, JWT-based authentication, Redis, Kafka, and an optional AI reflection layer. The key design decision is consent: normal analytics work without AI, and journal content is sent to AI only when the user explicitly enables it for that specific entry.

What problem does DKD solve?

People write journals but often do not get a clear view of their habits and emotional patterns. DKD turns entries into useful, privacy-aware information: streaks, consistency, sentiment breakdowns, weekly reflection, and optional AI-supported reflection.

What did you build personally?

I designed the APIs and backend flow: authentication, ownership checks, MongoDB models, journal CRUD, search and filters, dashboard calculations, analytics, reflection prompts, AI consent flow, weekly email reporting, caching, Kafka integration, Swagger documentation, and unit tests. A separate client can consume these APIs; I describe it broadly as the frontend or client application rather than claiming frontend expertise.

Interview tip: say “I designed” only for decisions you can explain. Say “I learned and implemented” for technologies you are still growing into.

2. Architecture in simple words

Layer	Responsibility
Client	Sends HTTPS requests with a JWT. It could be a web or mobile frontend.
Controller	Receives requests, gets the authenticated username, validates input, and returns DTOs.
Service	Contains business rules: ownership, calculations, AI orchestration, and report building.
Repository	Reads and writes MongoDB documents through Spring Data.
External services	Weather API, OpenAI API, email provider, Redis, and Kafka are kept separate from core journal logic.

Explain the request flow.

The client calls an API. Spring Security first checks the JWT. The controller receives the authenticated user, calls the correct service, and returns a clean response DTO. The service applies the business rules and uses repositories to work with MongoDB. This keeps controllers thin and makes the code easier to test.

Why do you use DTOs?

DTOs keep the API contract clean. For example, MongoDB uses ObjectId internally, but the client receives a normal 24-character string ID. DTOs also prevent exposing internal fields such as passwords, roles, configuration values, or database-specific details.

Why is the controller thin?

A controller should coordinate HTTP work, not hold business logic. If the logic stays in services, I can reuse it for a scheduler or Kafka consumer and test it without a web request.

3. Registration, login, and JWT

Explain registration.

The user sends username, email, and password. The backend validates the request, checks that the username is unique, hashes the password using BCrypt, assigns the normal USER role, and saves the user. The raw password is never stored.

Explain login and JWT in very simple language.

The user logs in with username and password. Spring Security verifies the password hash. If it is correct, the backend creates a signed JWT token. The client stores that token and sends it in the Authorization header on later requests. The JWT filter validates it and places the user identity into Spring Security, so each protected API knows who is calling it.

Why JWT instead of server sessions?

JWT works well for a separate client and a REST API because the server does not need to store a session for every user. The token is verified on each request. For stronger production security, I would use short-lived access tokens, refresh tokens, secure HTTP-only cookies where appropriate, and token revocation strategy.

How do you make sure one user cannot read another user’s journal?

I never trust a username or owner sent by the client. I get the current username from the JWT security context, load that user, and verify that the requested journal belongs to that user before reading, updating, deleting, or generating AI insight. This is enforced in the service layer.

4. Journal data and MongoDB

Why did you choose MongoDB?

Journal entries naturally have flexible fields, such as title, content, tags, selected sentiment, reflection prompt, and optional AI information. MongoDB lets me model that document-oriented data easily and iterate quickly. It also works well with Spring Data MongoDB. I still use clear boundaries and indexes, because NoSQL does not remove the need for good data design.

How is the data organized?

Users are stored in a users collection. Journal entries are separate Mongo documents and are referenced from the user using DBRef. AI insights and aggregate AI analytics insights are stored separately. This keeps journal ownership clear and lets insights be reused without changing the journal content.

How will you find a user quickly when the product grows?

Username has a unique index, so login and user lookup do not scan all users. For production scale, I would add indexes based on real queries, such as journal owner plus date, and review query plans. I would also avoid loading a very large list of journal entries at once by adding pagination and more database-side filtering.

Why not SQL?

A relational database would also be a valid choice. I chose MongoDB because the project is document-oriented and it helped me move quickly. If I needed complex joins, strict multi-table reporting, or financial-grade relational consistency, I would consider PostgreSQL.

5. Journal APIs, search, and analytics

What happens when a user creates a journal?

After JWT authentication, the backend validates the journal, saves the real MongoDB document, links it to the authenticated user, and returns a response DTO. The response includes the persisted ObjectId as a normal hex string, so the client can immediately use it for get-by-id or optional AI generation.

How do search and filters work?

Search is limited to the journal title, case-insensitive, with partial matches. A user can additionally filter by one sentiment and a date range, then sort by date ascending or descending. All filters are optional and can be combined. The search service works only with the authenticated user’s entries.

What is in weekly analytics?

The week starts on Monday and ends today, not a future Sunday. I calculate total entries, unique journaling days, current streak, dominant sentiment, sentiment breakdown, average entries per elapsed day, and consistency. Consistency counts unique days, so multiple entries on one day still count as one journaling day.

How do you calculate a streak?

I put unique entry dates into a set and start from today. While today, yesterday, and earlier consecutive dates exist, I increment the streak. A set prevents two entries on the same day from incorrectly increasing the streak.

6. Privacy and consent - your strongest answer

Your app has journals. How do you protect privacy?

Privacy is a central design concern. Passwords are hashed. APIs are protected by JWT. Every journal operation checks ownership. I return DTOs instead of exposing internal entities. Most importantly, AI is opt-in and separated into two consent levels: global AI analytics and per-journal content AI.

Does the app owner read journal content?

The application code does not provide an owner-facing endpoint to browse user journals. Access is controlled by the user's JWT and ownership checks. However, I would not claim end-to-end encryption: the backend database can technically process plaintext journal content, and an external AI provider receives content only for entries where the user explicitly enables content-level AI. In a future version, I would add client-side or field-level encryption, key management, audit logs, and a clear privacy policy.

What is the difference between global AI and per-journal AI?

Global AI is for a broader recent pattern. It sends metadata only: selected sentiments, dates, titles, reflection prompts, and consistency. It does not send journal content. Per-journal AI is separate: the user must enable AI for that specific journal before its content is sent to OpenAI for a summary and supportive reflection.

Why is that better than enabling AI for everything?

It gives the user control. They can get normal analytics without AI, obtain high-level pattern insight with metadata only, and selectively choose a sensitive entry for content-level analysis. That is a better privacy product decision than silently sending every entry.

7. AI workflow and cost control

Explain the individual journal AI flow.

The user enables global AI in settings and also enables AI for a specific journal. The backend verifies both permissions and journal ownership. It sends structured context to OpenAI: title, selected sentiment, tags, reflection prompt, and content. The model returns JSON with a summary, detected sentiment, positive observation, reflection question, and encouragement. The result is parsed, saved as an AIInsight, and reused later instead of regenerating it.

Explain the global AI analytics flow.

When the user opts into global AI, DKD can generate a recent 30-day insight from metadata only. It summarizes patterns using entries, unique journaling days, sentiment counts, titles, and reflection prompts. The result is saved as AIAnalyticsInsight and appears in analytics. It can also be reused in the weekly email. Normal analytics still work even when AI is off.

How many OpenAI calls happen?

Signup and login make zero OpenAI calls. Opening the dashboard or analytics GET endpoint makes zero OpenAI calls. A per-journal AI request can make one call only after explicit consent. A global AI insight can make one call when generated. Existing saved insights are reused for the same recent period and entry count, which avoids repeated calls during refreshes. The weekly email does not call OpenAI; it reuses stored insights.

How would you prevent duplicate simultaneous AI calls at scale?

The current design reuses saved results, which solves normal repeat requests. For multiple app instances or simultaneous clicks, I would add a database-level generation status or distributed lock before calling OpenAI, with a unique key for user and analysis window. That prevents two requests from generating the same insight at the same time.

Is this a therapist or medical tool?

No. The prompt explicitly asks for supportive, non-medical reflection. It does not diagnose, prescribe, or give therapy. If the product handled crisis content, I would add a dedicated safety flow and human-reviewed policy rather than relying on general AI output.

8. Weekly email, scheduler, Kafka, and fallback

What does the weekly email contain?

It includes current streak, entries this week, consistency, dominant sentiment, and average entries per day. If there are stored AI insights, it can include their weekly summary and the latest positive observation, reflection question, and encouragement. It does not call OpenAI while sending the email.

How is the weekly email triggered?

A Spring scheduler runs on Sunday and finds eligible users. For each user, it builds the report from stored journal data, weekly analytics, and stored AI insights. The feature is designed so the report can be delivered through Kafka rather than blocking the scheduler.

Why Kafka?

Kafka decouples producing a weekly report request from sending the email. The producer can publish an event and the consumer handles delivery. This makes the system more resilient and easier to scale because email work can be retried independently.

What is your Kafka fallback?

The application preserves a fallback path so the report can still be handled if Kafka is not available. This avoids losing the user-facing weekly report because one infrastructure component is down. In a larger production system I would also add retry policies, dead-letter topics, idempotency keys, and monitoring.

9. Redis, configuration, and external APIs

Where do you use Redis?

Redis is used as a cache for configuration that is read repeatedly, such as the AI system prompt. The application first checks Redis. On a cache miss it loads the value from the MongoDB configuration collection, stores it in Redis, and then uses it. This reduces repeated database reads.

Why is the AI prompt configurable?

The system prompt affects AI behavior. By keeping it in configuration rather than hardcoding it, I can update wording and safety instructions without a code deployment. The application’s configuration cache helps reduce read overhead.

How do you handle weather?

Weather is an optional external API integration. I return a structured weather response to the client rather than an unstructured string, making it easy for a dashboard to show location, condition, and temperature cleanly. External API errors should be handled without breaking the core journaling experience.

10. Admin, Swagger, testing, and code quality

What is the admin role?

Admin is a separate role for managing application-level resources such as reflection prompts. It is not a role that should casually read user journal content. Sensitive administration should have its own authorization checks and audit trail in a production system.

Why reflection prompts?

Prompts help a user start writing and encourage more thoughtful journaling. Active prompts are selected deterministically for the day, so every user sees the same prompt during that day instead of a different prompt on every refresh. Deleting a prompt is a soft delete: it is marked inactive to preserve history.

Why Swagger?

Swagger/OpenAPI documents the backend contract. It lets a frontend developer or tester see endpoints, request shapes, authentication requirements, and expected responses without reading all the code.

How did you test?

I wrote focused unit tests for service calculations and important business rules: search filters, dashboard values, analytics boundaries, streaks, reflection prompt behavior, AI insight behavior, email reporting, JWT utility behavior, and response DTO mapping. I also use API testing for end-to-end flows such as login, create journal, get by returned ID, and optional AI generation.

What code-quality decisions did you make?

I separate controllers, services, repositories, entities, and DTOs. Search logic is in a dedicated service. Security username retrieval is centralized in SecurityUtil. I keep calculations in reusable helper methods rather than one long controller or service method.

11. Scalability questions

What happens if you have many users?

I would scale the Spring Boot application horizontally behind a load balancer, keep it stateless through JWT, use MongoDB indexes and pagination, move slow work such as emails and AI to Kafka consumers, cache suitable configuration with Redis, and add observability for latency, failures, and queue backlog.

What are the current limitations?

The current project loads a user's referenced journals for some calculations, so very large journals need database-side aggregation or pagination. AI duplicate protection is good for normal repeat requests but needs a distributed lock or generation-status record for simultaneous requests across servers. Privacy can be strengthened with encryption and audit logs. These are known next steps, not things I claim are already solved.

How would you improve the database design later?

I would first measure. Likely steps are pagination for journal lists, indexes on frequently filtered data, an owner-and-date query path, and possibly a more direct journal repository query. I would only redesign references or embedding after understanding real data size and access patterns.

How would you deploy it?

I would containerize the service, provide environment-specific secrets outside source control, use managed MongoDB and Redis, use a managed Kafka option or a reliable message service, configure email credentials securely, enable HTTPS, health checks, logs, and monitoring. The client can be deployed separately and point to the backend URL.

12. API calls - concise answer

How many API calls are made in daily use?

Signup is one backend call. Login is one backend call that returns JWT. Creating a journal is one call. Opening a dashboard may call dashboard, analytics, reflection prompt, and weather APIs depending on what the client shows. Individual AI generation is one backend request and, only after consent, one OpenAI request. The weekly email is scheduler-driven, not a client API call.

How did you limit AI API calls?

AI is never called during signup, login, dashboard refresh, or normal analytics loading. It is an explicit action. Saved individual insights are returned again instead of regenerated. Saved recent analytics insights are reused for the same window and entry count. The weekly email reads stored results and makes no OpenAI call.

This is the short line to remember: “AI is opt-in, generated on demand, stored, and reused.”

13. Rapid-fire questions

What is an API?

A contract that lets the client ask the backend to perform a defined action or return data.

What does 401 mean?

The user is not authenticated or the JWT is missing/invalid.

What does 403 mean?

The user is authenticated but not allowed to perform that action.

What does 404 mean?

The requested resource does not exist, or is deliberately not exposed to this user.

Why BCrypt?

It is designed for passwords and includes salting and configurable work factor.

Why ObjectId response DTO?

MongoDB keeps ObjectId internally, while the client gets a stable string ID.

Why save AI output?

To avoid cost, delay, and inconsistent repeated AI answers.

What is idempotency here?

Repeating a request should not create duplicate or conflicting results; stored insight reuse helps.

What is a soft delete?

Marking data inactive rather than physically removing it.

Why not call AI from the scheduler?

Scheduled reports should be reliable and predictable; they reuse stored insights instead of depending on an external AI call.

14. Final interview strategy

• **Do not oversell.** Say what works today and clearly label future improvements.

• **Lead with the product decision.** For AI, lead with consent and privacy, then explain the technology.

• **Use one concrete flow.** Example: login - create journal - view analytics - optionally request AI insight - receive weekly report.

• **When stuck, return to first principles.** “The service gets the authenticated user, checks ownership, then applies the business rule.”

• **Talk about trade-offs.** MongoDB vs SQL, JWT vs session, sync vs Kafka, convenience vs privacy.

Best closing line

“DKD is not just a collection of technologies. The main design goal was to make journaling useful while keeping control with the user. Normal analytics work without AI; AI is optional, scoped, stored, and reused.”

Good luck, Tejas. You do not need to sound like a senior engineer. You need to sound like the engineer who thoughtfully built and understands this project.